

Chapter 6: Data Engineering and Automation

Advanced Python Programming For Data Science



Chapter Overview

This chapter focuses on:

- Understanding the role of **data engineering** in applied data science
- Designing and implementing **ETL (Extract, Transform, Load) pipelines**
- Automating data workflows using schedulers
- Ensuring reliability through logging, monitoring, and error handling
- Managing data storage and retrieval for pipelines
- Generating automated analytical reports

The emphasis is on **building reliable and automated data workflows**, not just performing one-time analysis.

Duration: 8 hours

Marks Weightage: 11 Marks



Learning Outcomes

After completing this chapter, students will be able to:

- Explain the role of data engineering in applied data science
- Design and implement basic ETL pipelines using Python
- Automate data workflows using schedulers such as CRON
- Apply logging, monitoring, and error handling in data pipelines
- Choose appropriate data storage and retrieval strategies
- Generate automated reports in formats such as Excel, HTML, and PDF
- Develop an end-to-end automated analytics pipeline



Chapter Structure

- 6.1** Overview of data engineering in applied data science
- 6.2** Designing and implementing ETL pipelines
- 6.3** Automating workflows with schedulers
- 6.4** Logging, monitoring, and error handling
- 6.5** Data storage and retrieval strategies
- 6.6** Automated report generation
- 6.7** Case study: End-to-end automated analytics pipeline

6.1 Overview of Data Engineering in Applied Data Science

What is Data Engineering?

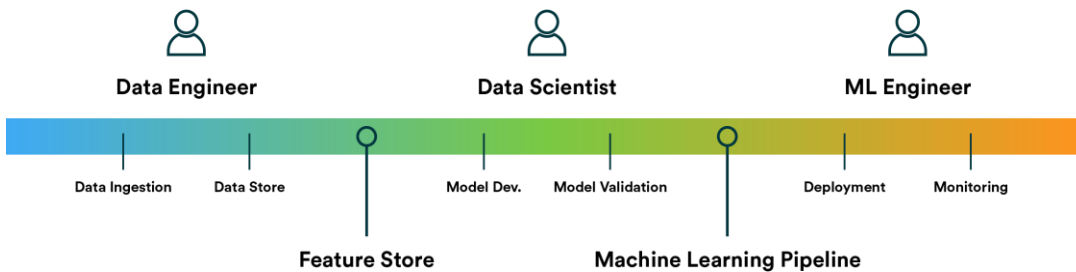
Data Engineering is the process of **designing, building, and maintaining** systems that collect, process, store, and deliver data reliably for analysis and decision-making.

It focuses on:

- Data pipelines
- Automation
- Scalability
- Reliability

This enables data scientists and analysts to work with **clean, ready-to-use data** instead of raw, unreliable data.

Data Engineering Focus



Data Engineering focuses on:

- Collecting data from multiple sources
- Cleaning, transforming, and storing data
- Making data **reliable, scalable, and available**

Key Difference

- Data Scientist → analyzes data
- Data Engineer → builds systems that deliver data

Data Scientist

Integration

Insights

Infrastructure

Data architecture
Infrastructure setup
Database management
Scalability solutions

Modeling

Machine learning
Predictive modeling
Statistical analysis
Algorithm optimization

Reporting

KPI tracking and performance benchmarking
Reporting automation and dashboard creation
Business, market, and industry analysis

Pipeline optimization
Data cleaning automation
Real-time data processing

Data querying
Data analysis
Insight generation

Data Engineer

Jeff Winter

Data Analyst

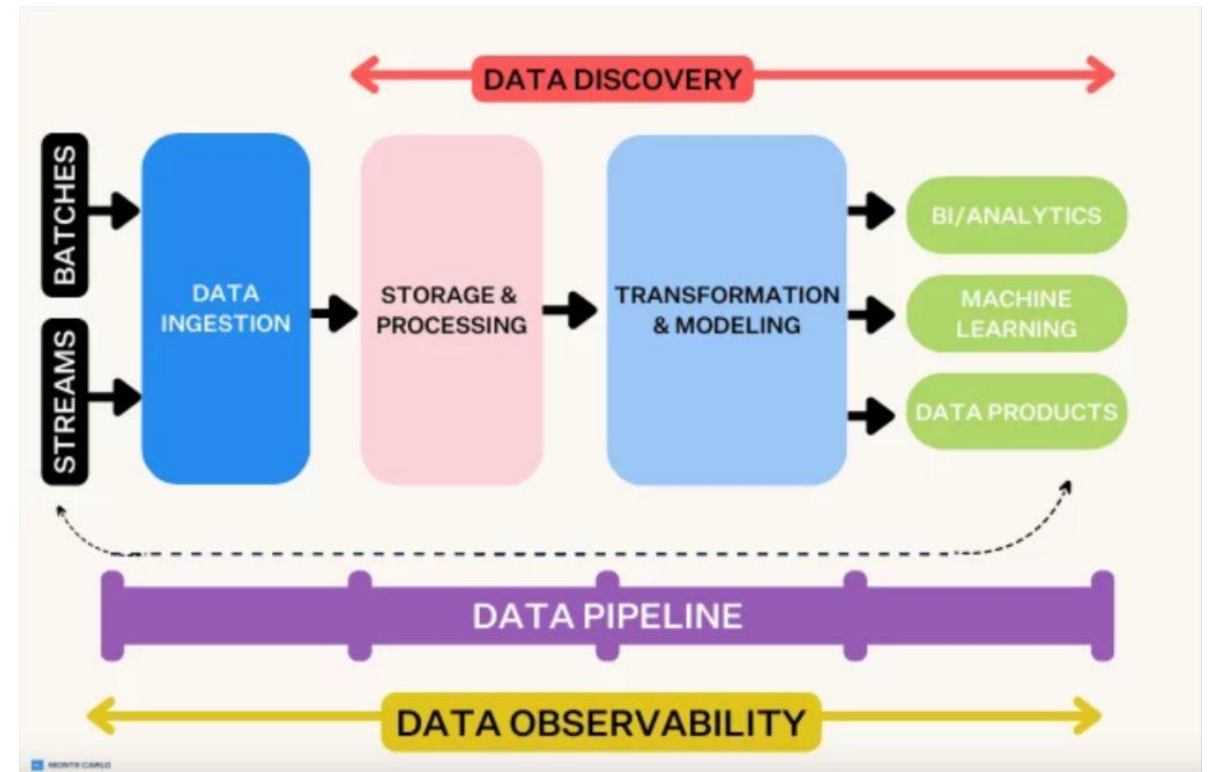
Why Data Engineering is Important

Without data engineering:

- Analysis becomes manual and slow
- Errors increase
- Systems fail at scale

With data engineering:

- Pipelines run automatically
- Data is consistent and reliable



6.2 Designing and Implementing ETL Pipelines

What is an ETL Pipeline?

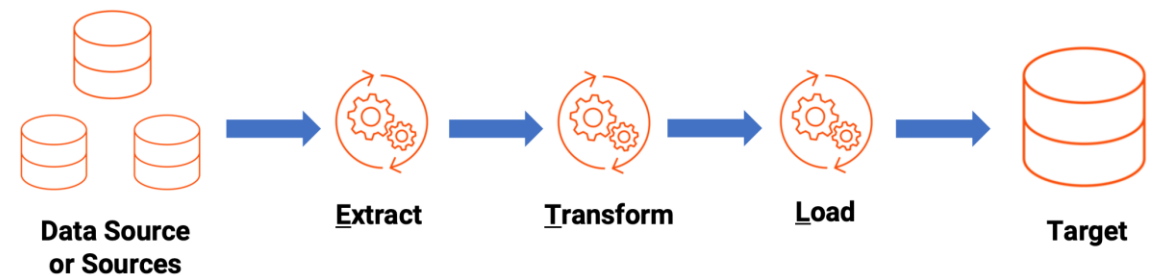
An **ETL Pipeline** is a structured process used to **move and prepare data** from multiple sources so it can be analyzed or reported.

ETL stands for:

- **Extract** – collect data from sources such as files, databases, or APIs
- **Transform** – clean, filter, and process the data
- **Load** – store the processed data in a database or data warehouse

Note: ETL pipelines are a **core component of data engineering** and automated data workflows.

How an ETL Pipeline Works



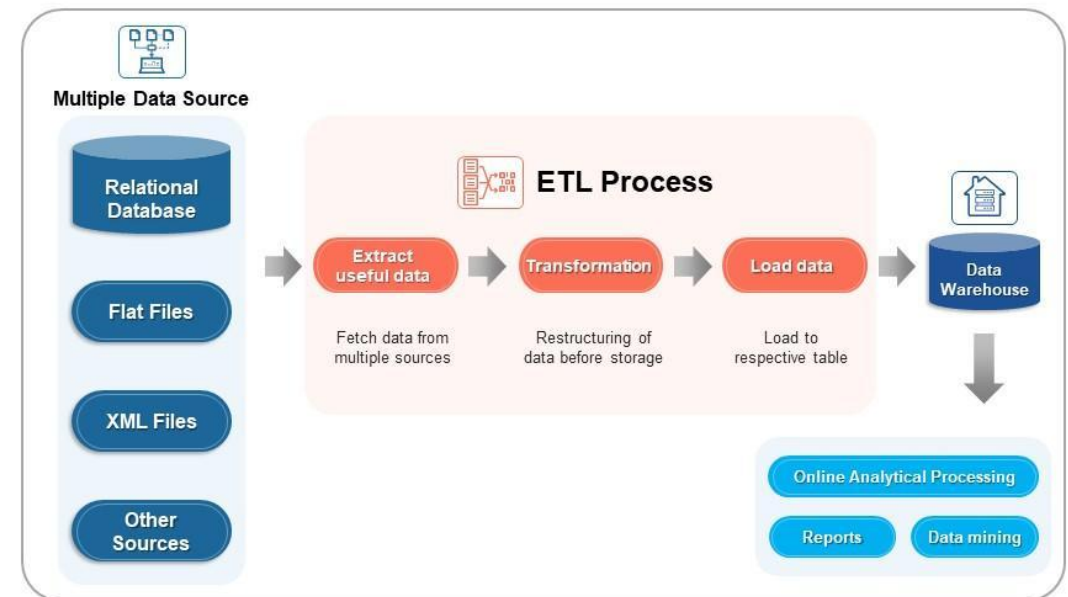
ETL Pipeline Components

Common Components

- Data sources (files, APIs, databases)
- Transformation logic
- Storage systems
- Scheduling and monitoring

Three Components of ETL Process Flow Model

This slide is 100% editable. Adapt it to your needs and capture your audience's attention.





6.3 Automating Workflows with Schedulers

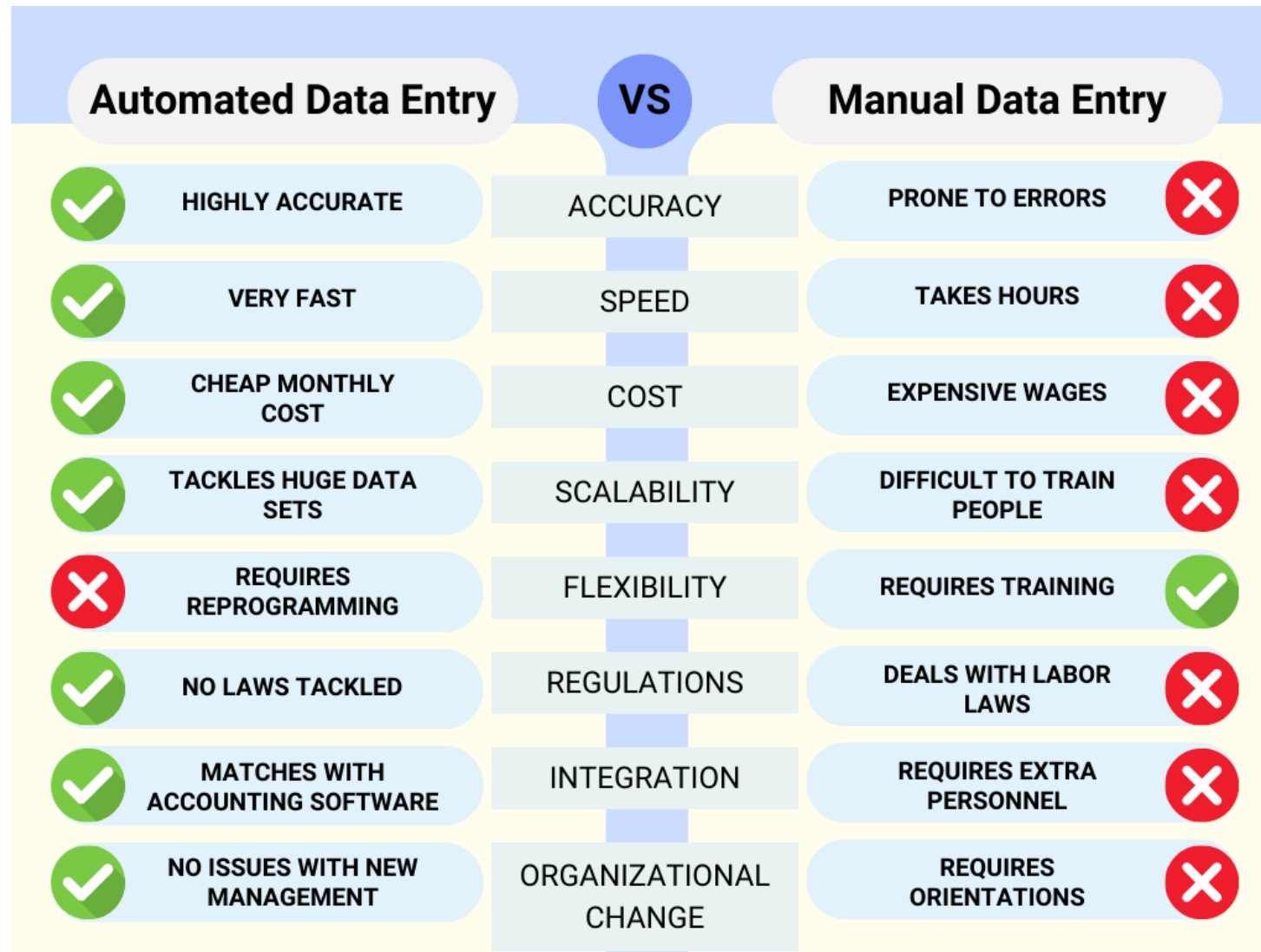
Why Automation is Needed?

Manual workflows:

- Time-consuming
- Error-prone

Automated workflows:

- Run on schedule
- Reduce human intervention



Automated Data Entry VS Manual Data Entry

Schedulers in Python

What is a Scheduler?

A scheduler is a system or tool that automatically runs a task at a specified time or at regular intervals without manual intervention.

Schedulers are used to:

- Run tasks **periodically** (daily, hourly, weekly)
- Automate repetitive jobs
- Ensure workflows execute on time

Example in data engineering:

- Refreshing data every night
- Running an ETL pipeline every hour
- Generating reports every week

Schedulers are a key part of **automation in data pipelines**, ensuring tasks run **reliably and consistently**.

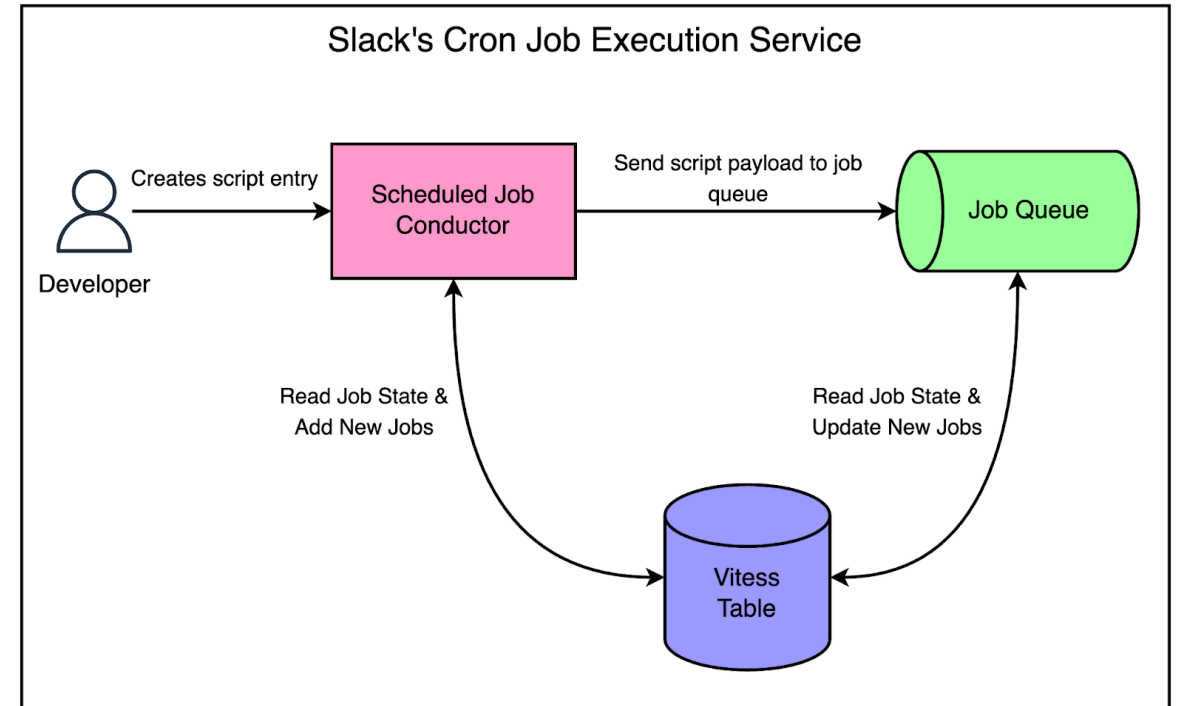
Tools for Schedulers

Common Tools

- CRON jobs
- Python scheduling libraries

Schedulers ensure pipelines run:

- Daily
- Hourly
- On demand



6.4 Logging, Monitoring, and Error Handling

Why Logging is Important?

Logging helps to:

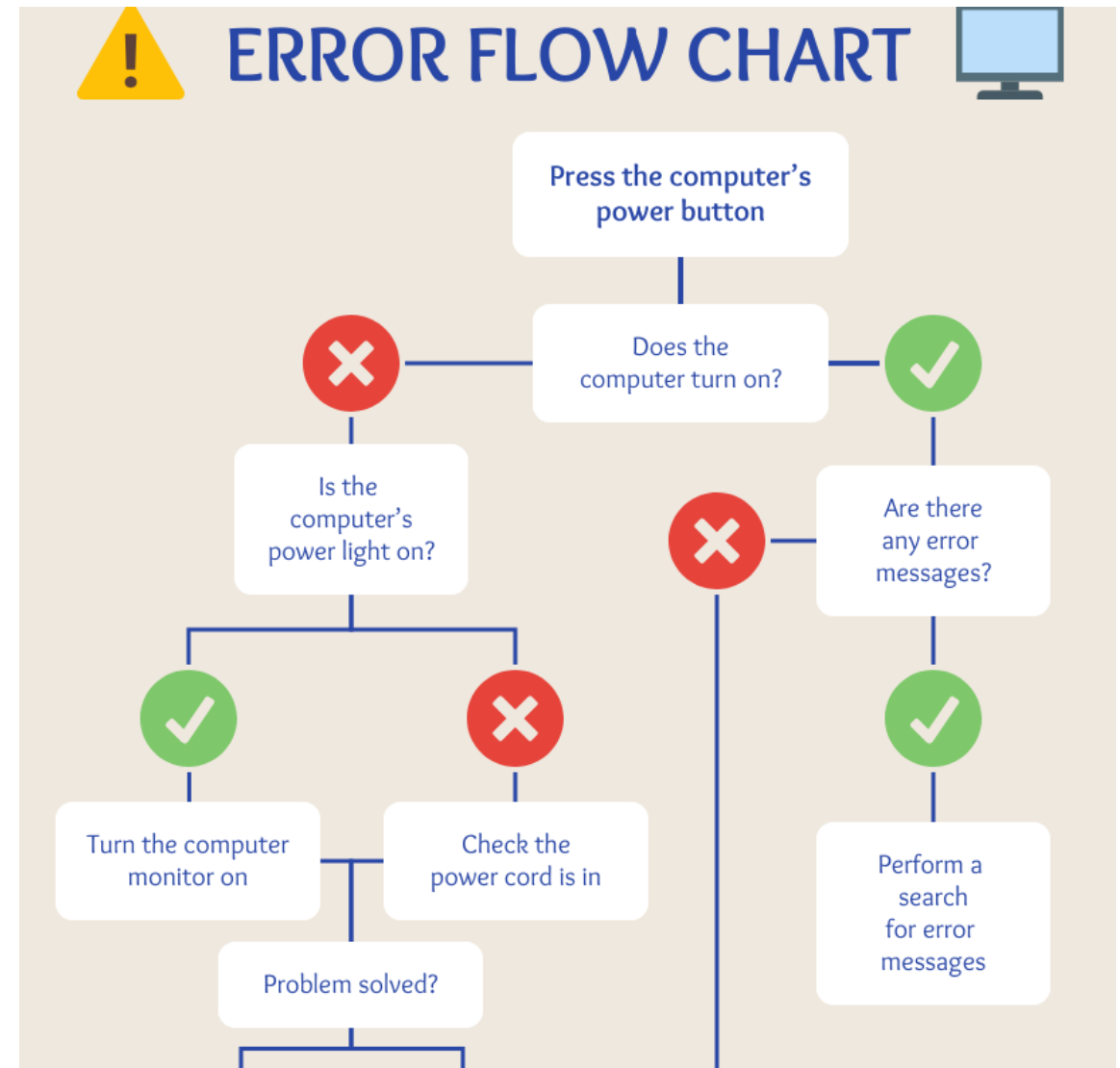
- Track pipeline execution
- Detect failures
- Debug issues quickly

Monitoring ensures pipelines are healthy.

Error Handling Strategies

- Try-except blocks
- Alerts and notifications
- Retry mechanisms

Reliable pipelines handle failures **gracefully**.





6.5 Data Storage and Retrieval Strategies

Common Storage Types

- Files (CSV, JSON)
- Relational databases
- NoSQL databases

Storage choice depends on:

- Data size
- Query needs
- Performance requirements

Types of Data Storage

Data storage refers to **how and where data is stored** so it can be accessed later for analysis and reporting.

1. File-Based Storage

- Examples: CSV, JSON, Excel, Text files
- Simple and easy to use
- Suitable for small datasets and local processing

2. Relational Database Storage

- Examples: MySQL, PostgreSQL, SQLite
- Data stored in tables (rows and columns)
- Supports structured queries using SQL

3. NoSQL Database Storage

- Examples: MongoDB, Cassandra
- Flexible schema
- Suitable for large, unstructured, or semi-structured data

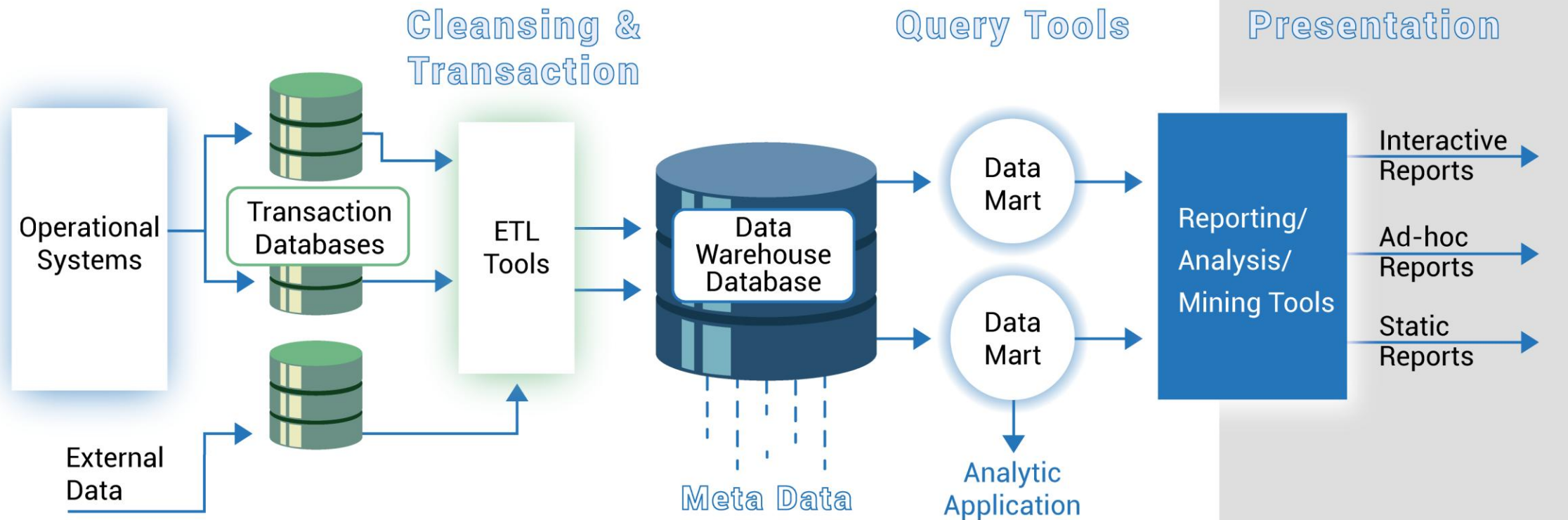
4. Data Warehouse Storage

- Designed for analytics and reporting
- Stores historical and aggregated data
- Optimized for fast read operations

5. Cloud Storage

- Examples: object storage services
- Scalable and accessible over the internet
- Commonly used in modern data pipelines

Data Warehouse Database



Data Retrieval Strategies

Data retrieval strategies define how stored data is accessed efficiently for processing, analysis, or reporting.

Common Retrieval Strategies

- **Full Retrieval**
Entire dataset is loaded at once
Used for small datasets
- **Query-Based Retrieval**
Only required records are fetched using queries
Improves performance and reduces memory usage
- **Incremental Retrieval**
Retrieves only new or updated data
Efficient for large and continuously growing datasets
- **Batch Retrieval**
Data is retrieved in fixed-size chunks
Prevents memory overload
- **On-Demand Retrieval**
Data is fetched only when requested
Used in dashboards and interactive reports

Choose the retrieval strategy based on **data size, frequency of access, and performance needs**.

6.6 Automated Report Generation

What is Automated Report Generation?

Automated report generation is the process of:

- Creating reports automatically from data
- Running on a schedule or trigger
- Delivering results without manual effort

Why Automated Reports Are Needed

Problems with Manual Reporting

- Time-consuming
- Error-prone
- Inconsistent

Benefits of Automation

- Saves time
- Ensures consistency
- Delivers reports on time
- Automated reports are essential in **production data systems**.

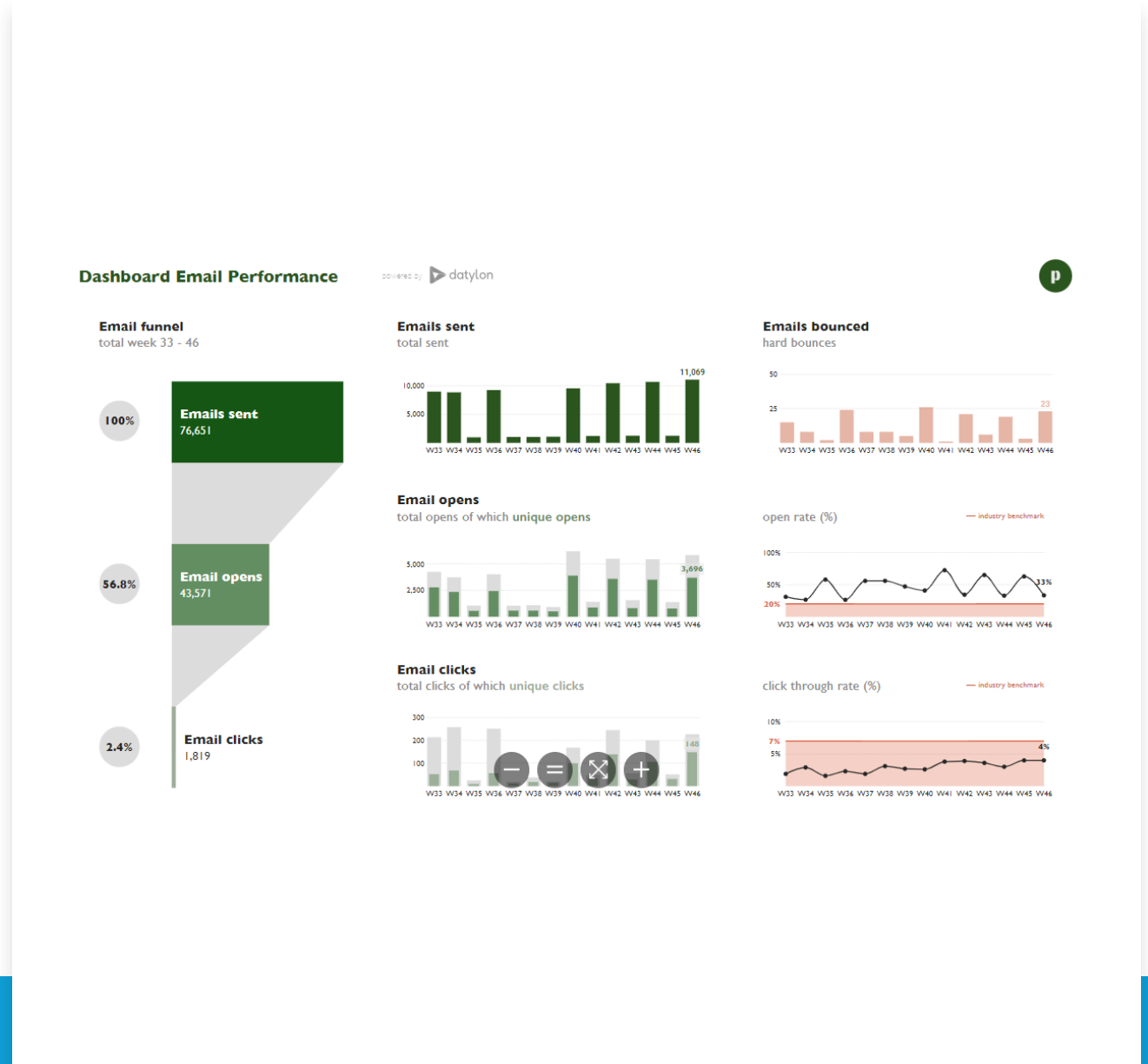
Common Report Formats

Popular Formats

- **Excel:** tabular data and summaries
- **PDF:** static, printable reports
- **HTML:** interactive, web-based reports

Format choice depends on:

- Audience
- Interactivity needs
- Distribution method



Automated Reporting Workflow

Typical Workflow

- Data is collected and processed
- Analysis is performed
- Visualizations are generated
- Report is created automatically
- Report is shared or stored

Real-World Use Cases

- Daily sales reports
- Weekly performance dashboards
- Monthly financial summaries

Automated reports ensure decisions are made using **fresh and reliable data**.



6.7 Case Study: End-to-End Automated Analytics Pipeline

Pipeline Steps

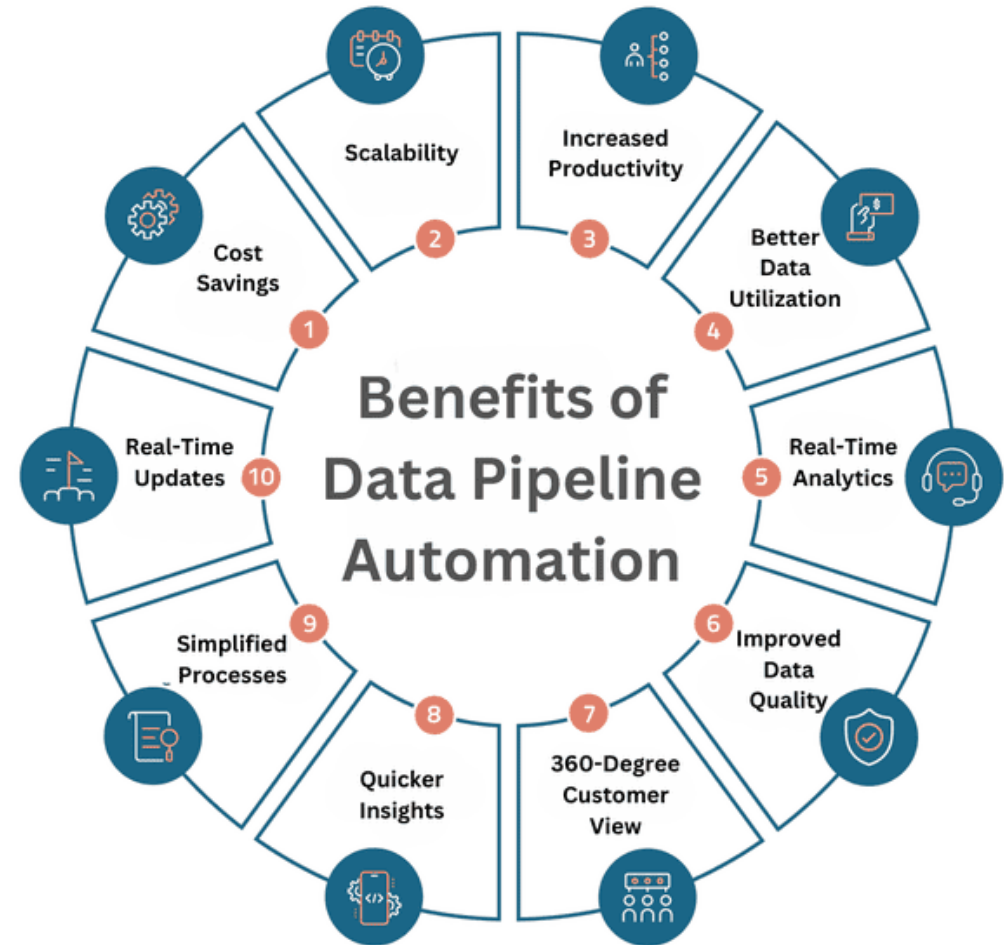
1. Data ingestion
2. Transformation
3. Storage
4. Automated analysis
5. Report generation

This case study integrates **all Chapter 6 concepts**.

Automation and Scheduling

- Pipeline runs on a schedule
- No manual execution required
- Ensures regular data updates

Automation is the **core strength** of this pipeline.

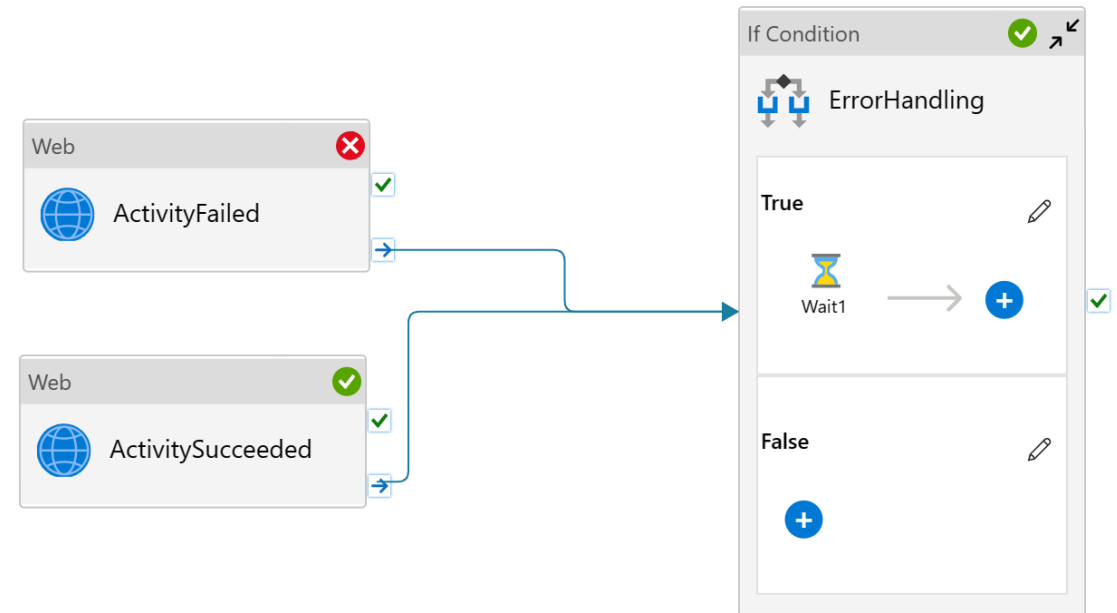


Monitoring and Error Handling

Key Practices

- Logging pipeline steps
- Detecting failures
- Retrying failed tasks
- Sending alerts

This ensures **pipeline reliability**.



Outcomes of the Case Study

- Design automated pipelines
- Combine ETL, scheduling, and reporting
- Build scalable data systems
- Apply data engineering concepts practically

This case study prepares students for **real-world data engineering and analytics roles**.

Chapter 6

Summary

- Data engineering builds **reliable and automated data workflows**
- ETL pipelines extract, transform, and load data
- Schedulers automate pipeline execution
- Logging and monitoring ensure pipeline reliability
- Storage and retrieval strategies manage data efficiently
- Automated reports deliver insights without manual effort